A Constructive Lean Contender Beyond Contender 5

Ramified W-reflection, higher-type feedback, and an explicit tower-height separation

Research prepared for Vladimir Reshetnikov
OpenAI

12 October 2026

Abstract

We propose a natural number $\mathfrak{N} = R_\Theta(64)$ for a Lean adaptation of the *name-the-biggest-number* contest. Its definition combines a finite maximum over a total language, recursion on countably branching well-founded trees, diagonalization through arbitrary finite function types, and feedback from previously defined numerical values into type-level indices. The construction does not refer to the preceding winning constant and does not use a halting oracle, unrestricted semantic maximization, or an assumed universal Lean evaluator.

We give an exact semantic port of the upstream Contender 5, including its behavior on malformed raw terms. A short quotation argument proves $C_5 < \mathfrak{N}$. A stronger argument, using a depth-20 oracle-iteration term, proves $E_{C_5+1}(64) < \mathfrak{N}$, where $E_0(x) = x$ and $E_{j+1}(x) = 2^{E_j(x)}$. We also establish explicit child-stage comparisons, a 2^{28}-branch visibility bound at input 64, and eventual domination of every fixed numerical program in a stage's own language.

The accompanying artifact contains Lean definition and proof source, a strict axiom/timing gate, and a Python small-instance model with 62,065 passing assertions. **The Lean source has not been compiler-checked: the preparation environment had no Lean toolchain and attempted downloads failed.** Consequently, kernel acceptance, the empty-axiom audit, and the contest's 15-second/60-second timing requirements remain unverified. The mathematical proposal and proofs are not presented as an already accepted contest entry.

Contents

1 What is being proposed, and what is being claimed 3

2 The upstream number, without changing its semantics 4

3 A stronger, but still total, finite language 6

4 Short witnesses and escaping a fixed language 8

5 Ramified reflection on trees 9

6 A quantitative amplification: oracle-fed iteration 10

7 Higher-type countable iteration 12

8 Numerical feedback into type-level indices 13

9 The exact comparison with Contender 5 14

10 Why the construction is constructive rather than a loophole 15

11 Lean artifact architecture 16

12 Validation performed and validation still required 17

13 Further research questions 18

14 Conclusion 19

A The central Lean definition, in ASCII notation 20

B Provenance and reproducibility notes 21

1 What is being proposed, and what is being claimed

The user asks for a serious constructive contender after replacing Coq with Lean in Cody Roux's contest. The source repository attributes Contender 5 to Samuel Grütter and defines it as the largest numerical output at weighted syntax depth 42 in a totalized simply typed language with natural-number recursion [1, 3]. The original rules concern *typechecking time*, not the time required to evaluate the resulting natural number. They also prohibit axioms and defining the next entry from the previous winning constant [2].

Our proposed declaration is

$$\boxed{\mathfrak{N} = R_\Theta(64).} \tag{1}$$

Every symbol in this equation will receive a constructive definition. In particular, R is a tree recursor, not an appeal to a vaguely specified fast-growing hierarchy. The central numerical comparison is

$$\boxed{E_{C_5+1}(64) < \mathfrak{N}, \qquad E_0(x) = x, \quad E_{j+1}(x) = 2^{E_j(x)}.} \tag{2}$$

The role of C_5 in (2) is *only in the theorem*; it is absent from the definition of $\mathfrak{N}$.

There is no claim of a mathematically greatest constructive natural number. Any fixed total definition admits further amplification. Nor do we assert a sharp proof-theoretic ordinal for this construction. The contribution is a particular ambitious, fully specified proposal with provable separation, carefully delimited formalization status, and reproducible artifacts. The general finite-language-maximization idea is inherited from the upstream entry. Our proposed combination of W-recursive object-language operations, ramified numerical oracles, type-index feedback, and explicit budgeted comparison witnesses is developed here; a comprehensive priority claim is not made.

1.1 A precise Lean adaptation of the rules

We retain the mathematical intent of the original contest and propose the following operational interpretation. The pinned target is Lean 4.35.0; pinning is necessary for reproducibility, not a claim that this source was successfully checked on that version [8].

1. The entry is a closed definition of type `Nat`, using only total computational definitions. It must not depend on a custom axiom, a proof placeholder, classical choice, noncomputable selection, an unsafe replacement, or an external numerical oracle.
2. The new definition dependency closure must compile in at most 15 seconds, and the comparison proof in at most 60 seconds, on a disclosed reasonable machine. Distributed standard-library artifacts may be prebuilt; new contestant modules may not be hidden in a precompiled cache.
3. The entry's definition must not refer to the previous winning *number*. Reusing an explicitly charged source language and its evaluator is allowed. The comparison proof may name the previous entry.
4. The transitive axiom reports for the entry and required comparison must be empty. Importing a library that contains unused axioms is not itself an axiom dependency; using one is.

Lean distinguishes computational definitions from additional principles such as classical choice and propositional extensionality. These distinctions make an actual transitive audit important [7]. The package's gate intentionally does not whitelist such principles. The surrounding explanatory

mathematics occasionally uses extensional descriptions of functions; those analysis-only identities are not imported into the strict Lean proof path.

1.2 Validation boundary

Three different statements must not be conflated: a complete mathematical proof, proposed formal proof source, and a successful kernel run. This report provides the first two for the basic strict comparison. It does not provide the third. The stronger tower theorem, the binary budget bounds, and the higher-type extensional identities are proved in this article and tested in small instances where appropriate; they are not claimed as completed Lean formalizations. Section 12 records the exact evidence.

2 The upstream number, without changing its semantics

2.1 Types, raw terms, and weighted depth

Let the old types be generated by

$$A, B ::= \mathsf{N} \mid A \to B, \qquad [\![\mathsf{N}]\!] = \mathbb{N}, \qquad [\![A \to B]\!] = [\![A]\!] \to [\![B]\!].$$

Their depth is $\mathrm{td}(\mathsf{N}) = 1$ and $\mathrm{td}(A \to B) = 1 + \max(\mathrm{td}\, A, \mathrm{td}\, B)$. The raw terms are

$$t ::= \mathsf{var}(i) \mid \lambda_{A,B} t \mid t\, u \mid \mathsf{0} \mid \mathsf{S} \mid \mathsf{rec}_A.$$

The recursor has semantic type

$$\mathsf{rec}_A : [\![A]\!] \to (\mathbb{N} \to [\![A]\!] \to [\![A]\!]) \to \mathbb{N} \to [\![A]\!].$$

The weighted term depth is

$$\begin{aligned} \mathrm{depth}(\mathsf{var}(i)) &= i + 2, \\ \mathrm{depth}(\lambda_{A,B} t) &= 1 + \max(\mathrm{td}\, A, \mathrm{td}\, B, \mathrm{depth}\, t), \\ \mathrm{depth}(tu) &= 1 + \max(\mathrm{depth}\, t, \mathrm{depth}\, u), \\ \mathrm{depth}(\mathsf{0}) = \mathrm{depth}(\mathsf{S}) &= 1, \qquad \mathrm{depth}(\mathsf{rec}_A) = 1 + \mathrm{td}\, A. \end{aligned}$$

In particular, depth counts type annotations and variable labels. It is not ordinary node count or source-byte length.

2.2 The significance of malformed terms

The upstream evaluator does *not* first discard terms that fail a typing judgment. It returns a tagged semantic value for every raw term. This is visible directly in `interp_term`, `cast_impl`, and `error` in the source [3]. Replacing this evaluator with a well-typed-term enumerator would silently change the target of comparison.

For each type define its default value by

$$\bot_{\mathsf{N}} = 0, \qquad \bot_{A \to B}(x) = \bot_B.$$

A computational cast pair between two types is identity at matching natural types. At matching arrow shapes with syntactically equal component types it uses recursively defined contravariant input and covariant output maps. If these tests fail, both maps return the appropriate default value. Thus no opaque equality proof is needed to run a cast.

An environment is a list of tagged values. Variable labels are absolute binder levels: label 0 refers to the outermost binder, not the newest one. For an environment of length l, lookup of $i < l$ uses position $l - 1 - i$ in the cons-based list; otherwise lookup fails and produces the tagged value $(\mathsf{N}, 0)$. A lambda creates a semantic function, evaluates its body under a new tagged argument, and casts the body's result to its stated codomain. Application of a tagged function casts its argument to the function's domain. Application of a nonfunction produces $(\mathsf{N}, 0)$.

Finally, $\mathrm{val}_0(t)$ evaluates t in the empty environment and returns its natural component if its tag is N; otherwise it returns zero. This includes raw terms with free variable labels: "empty-environment interpretation" is not a promise of syntactic closedness. For example, $\mathrm{val}_0(\mathsf{S}\,\mathsf{S}) = 1$, since the function-valued argument is cast to natural zero rather than rejected.

2.3 The finite enumerator and the selected maximum

Let $U_0 = []$ and

$$U_{d+1} = [\mathsf{N}] +\!\!+ [A \to B \mid A, B \in U_d].$$

Let $T_0 = []$. The list T_{d+1} concatenates, in the upstream order,

$$\begin{aligned}
&[\mathsf{var}(i) \mid i = d-1, d-2, \ldots, 0],\\
&[\lambda_{A,B} t \mid A, B \in U_d,\ t \in T_d],\\
&[tu \mid t, u \in T_d], \quad [\mathsf{0}], \quad [\mathsf{S}], \quad [\mathsf{rec}_A \mid A \in U_d].
\end{aligned}$$

The variable list is empty for $d = 0$. Induction establishes that membership in T_d is equivalent to having depth at most d; the upstream source proves this exact characterization [3].

For fidelity, the Lean baseline uses the original argmax algorithm, not just an independently specified maximum. Start with current value 0 and current best term $\mathsf{0}$, scan the list, and replace the best term exactly when a strictly larger value appears. Write the resulting selected term as

$$t_* = \mathsf{maxBy}(\mathrm{val}_0, 0, \mathsf{0}, T_{42}), \qquad C_5 = \mathrm{val}_0(t_*).$$

Lemma 2.1 (Selection and maximum)**.** *The term t_* belongs to T_{42}, and $C_5 = \max(\{0\} \cup \{\mathrm{val}_0(t) : t \in T_{42}\})$.*

Proof. At each scan step the selected term is either the old seed or a term already visited. The seed $\mathsf{0}$ belongs to T_{42}, so the final selection is in that list. The invariant "current value equals the selected value and is the maximum of the seed value and all visited values" holds initially and is preserved by either branch of the strict comparison. It yields the displayed identity after the final element. □

Proposition 2.2 (Mathematical fidelity of the port)**.** *The computational definitions in `Legacy.lean`, together with `Baseline.lean`, denote the same natural number as the pinned upstream `contender_5`.*

Proof. Match type constructors, raw constructors, natural numbers, and lists componentwise. Relate semantic natural values by equality and arrow values by preservation of this relation on related inputs. Structural induction on types shows that defaults are related and that both components of

each cast pair preserve the relation. The same Boolean comparisons select the same cast branches. The variable lookup functions use identical lengths, subtraction, and zero-based list positions.

Now induct on the raw term, with the induction hypothesis quantified over related environments. The lambda case extends those environments by related arguments and uses the codomain cast lemma. The application case first uses the two term hypotheses, then the domain cast lemma. Natural recursion preserves the relation by induction on its natural argument. Constants and failed lookups are immediate. Thus empty-environment numerical observations agree for every raw term. The two list enumerators agree by induction on the depth. Finally, the argmax scans use the same strict comparison on equal natural values, start from corresponding seeds, and make corresponding choices. Their numerical results therefore agree. □

This is a semantic port argument, not a claim of a machine-checked Coq-to-Lean translation certificate. The upstream Coq project was not compiled in this environment. The checked-in provenance pins both its commit and the exact source blob.

3 A stronger, but still total, finite language

3.1 The tree type

Define the strictly positive inductive type

$$\mathcal{W} ::= \mathbf{0} \mid \mathsf{sup}(\mathbb{N} \to \mathcal{W}). \tag{3}$$

An element is a countably branching well-founded tree. Its branch function may be represented by a finite program; it is not an explicitly stored infinite list. The constructor sup is *not* an ordinal supremum and we impose no quotient identifying trees of equal ordinal rank.

For any result type X, its recursor satisfies

$$\mathsf{fold}_X(z, s, \mathbf{0}) = z, \tag{4}$$

$$\mathsf{fold}_X(z, s, \mathsf{sup} f) = s\big(f,\ i \mapsto \mathsf{fold}_X(z, s, f(i))\big). \tag{5}$$

The recursive values are supplied as a function of the child index, not computed by eager traversal of every child. Strict positivity and recursors are the relevant Lean mechanisms [5, 6].

3.2 Types and primitives

The new type language is

$$A, B ::= \mathsf{N} \mid \mathsf{W} \mid A \to B, \qquad [\![\mathsf{W}]\!] = \mathcal{W}.$$

There is a natural recursor and a tree recursor at *every* result type in this finite grammar. For a fixed binary function $q : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$, the additional primitives are

$$\begin{aligned} \mathsf{treeZero} &: \mathcal{W}, & \mathsf{treeSup} &: (\mathbb{N} \to \mathcal{W}) \to \mathcal{W}, \\ \mathsf{oracle} &: \mathbb{N} \to \mathbb{N} \to \mathbb{N}, & [\![\mathsf{oracle}]\!] &= q, \\ \mathsf{treeRec}_A &: [\![A]\!] \to ((\mathbb{N} \to \mathcal{W}) \to (\mathbb{N} \to [\![A]\!]) \to [\![A]\!]) \to \mathcal{W} \to [\![A]\!]. \end{aligned}$$

The ordinary zero, successor, variables, lambdas, applications, and natural recursors are retained. Defaults and computational casts are extended to W by $\bot_{\mathsf{W}} = \mathbf{0}$ and identity on matching tree types. Nonfunction application again gives tagged natural zero.

There is also a constructor $\mathsf{quote}(t)$ for *any raw old term* t. It evaluates t in the old empty environment and returns the tagged natural $\mathrm{val}_0(t)$. Its depth charge is $1 + \mathrm{depth}(t)$. This is an explicit embedding of the old numerical semantics, not a constant for the old winning number, not a call to the old maximum, and not a free abbreviation for an arbitrarily large source file.

Let $\mathrm{val}_q(e)$ denote the new empty-environment natural observation. Every raw new term has an observation; no undecidable validity filter is inserted. The only oracle during this section is a *parameter*. In the eventual entry every such parameter is replaced by an already structurally smaller, total function.

3.3 Depth and exact finite enumeration

Both ground types have type depth 1; arrow depth is unchanged. Ordinary term depths are unchanged, all five unannotated constants $\mathsf{0}, \mathsf{S}, \mathsf{treeZero}, \mathsf{treeSup}, \mathsf{oracle}$ have depth 1, each recursor annotated by A has depth $1 + \mathrm{td}\, A$, and

$$\mathrm{depth}(\mathsf{quote}(t)) = 1 + \mathrm{depth}(t).$$

The new type list is

$$V_0 = [], \qquad V_{d+1} = [\mathsf{N}, \mathsf{W}] +\!\!+ [A \to B \mid A, B \in V_d].$$

The new term list $\mathcal{T}_0 = []$ and $\mathcal{T}_{d+1}$ is the concatenation of the following eight blocks:

$$\begin{aligned} &\mathcal{T}_d; \quad [\mathsf{0}, \mathsf{S}, \mathsf{treeZero}, \mathsf{treeSup}, \mathsf{oracle}]; \quad [\mathsf{var}(i) : i < d]; \\ &[\lambda_{A,B} e : A, B \in V_d,\ e \in \mathcal{T}_d]; \quad [ef : e, f \in \mathcal{T}_d]; \\ &[\mathsf{rec}_A : A \in V_d]; \quad [\mathsf{treeRec}_A : A \in V_d]; \quad [\mathsf{quote}(t) : t \in T_d]. \end{aligned} \tag{6}$$

The order of variable labels in the implementation is descending; the other products use list order. The first block deliberately makes the enumerator cumulative. Repetition changes execution cost but never changes a maximum.

Lemma 3.1 (Coverage)**.** *The list $\mathcal{T}_d$ is finite and its elements are exactly the raw terms of depth at most d. Moreover, $\mathcal{T}_d$ is included, with occurrences, in $\mathcal{T}_{d+1}$.*

Proof. Finiteness follows by induction, since each step uses finite products, maps, and concatenations. For types, induction gives $A \in V_d \iff \mathrm{td}\, A \le d$. For terms at step $d+1$, inspect the outer constructor. An application has both children in $\mathcal{T}_d$ exactly when its depth is at most $d+1$; a lambda additionally needs both annotations in V_d. The analogous equivalences handle recursors, variable labels, and quotation via the old enumerator. Constants occur at every positive step. The carried-over block cannot introduce a term of excessive depth. These observations prove both directions of coverage inductively. The last claim is immediate from the first block. □

The strict Lean comparison uses direct occurrence lemmas for the particular constructors it needs, so it does not depend on a formalized converse to this coverage lemma.

Definition 3.2 (Finite semantic envelope)**.** For a total binary q, define

$$M_q(d) = \max(\{0\} \cup \{\mathrm{val}_q(e) : e \in \mathcal{T}_d\}). \tag{7}$$

The implementation uses a recursive list maximum, not classical selection.

Proposition 3.3 (Totality and monotonicity)**.** *For every total computable q, M_q is total computable, and $M_q(d) \le M_q(d+1)$. Every occurrence $e \in \mathcal{T}_d$ satisfies $\mathrm{val}_q(e) \le M_q(d)$.*

Proof. The interpreter recurses on raw syntax. Semantic recursion inside its primitive constants is natural recursion or the tree recursor (4). Both have total input functions of the indicated types. Thus every list element has a natural observation. A finite maximum of these observations is computed by list recursion. Occurrence bounds follow by induction on the maximum computation, and monotonicity follows from the cumulative block in (6). □

The theorem is a totality statement, not an efficiency statement. In particular, evaluating $M_q(68)$ is not part of the proposed validation run.

4 Short witnesses and escaping a fixed language

4.1 Unary witnesses

Write $\overline{n} = \mathsf{S}^n\mathsf{0}$ for the unary numeral. Then $\mathrm{depth}(\overline{n}) = n+1$ and $\mathrm{val}_q(\overline{n}) = n$. For $i \le n$, the raw term

$$w_{i,n} = \mathsf{S}(\mathsf{oracle}\,\overline{i}\,\overline{n}) \tag{8}$$

has depth $\max(i+4, n+3) \le n+4$ and value $q(i,n)+1$. Therefore

$$q(i,n) < M_q(n+4) \quad (i \le n). \tag{9}$$

Likewise, the numeral $\overline{n+1}$ proves

$$n < M_q(n+4). \tag{10}$$

These statements need no monotonicity assumption on q.

4.2 Binary numerals with an explicit syntax cost

Because variable names are absolute binder levels, embedding a term below new binders requires care. At top level define the depth-8 term

$$D = \mathsf{rec_N}\,\mathsf{0}\,(\lambda_{\mathsf{N},\mathsf{N}\to\mathsf{N}}\lambda_{\mathsf{N},\mathsf{N}}\mathsf{S}(\mathsf{S}(\mathsf{var}(1)))). \tag{11}$$

It denotes the function $x \mapsto 2x$. The outer step argument has level 0; the accumulator has level 1. The natural-recursion equations prove the claim by induction on the input.

Let $b(0) = \mathsf{0}$. For $n > 0$, start with $\mathsf{0}$ and read the ordinary binary digits of n from left to right, replacing the current term e by De for digit 0 and by $\mathsf{S}(De)$ for digit 1. These occurrences of D are all in the empty environment. With $\mathrm{bits}(0) = 0$ and $\mathrm{bits}(n) = \lfloor \log_2 n \rfloor + 1$ for $n > 0$, we have

$$\mathrm{val}_q(b(n)) = n, \qquad \mathrm{depth}(b(n)) \le 8 + 2\,\mathrm{bits}(n). \tag{12}$$

The value claim follows from binary positional notation. The first digit creates depth at most 10; every later digit increases depth by at most two. The displayed bound also covers zero.

Theorem 4.1 (Explicit escape from a fixed oracle language)**.** *Fix q. Let p be any fixed raw term whose empty-environment interpretation has type $\mathsf{N} \to \mathsf{N}$ and denotes a function $f : \mathbb{N} \to \mathbb{N}$. Put $c = \mathrm{depth}(p)$. Then*

$$f(n) < M_q(n) \qquad \text{for every } n \geq \max(c+2, 32). \tag{13}$$

Consequently, M_q is not denoted by any such fixed unary term in its own q-language.

Proof. The term $\mathsf{S}(p\, b(n))$ has value $f(n)+1$ and depth at most $2 + \max(c, 8 + 2\,\mathrm{bits}(n))$. For $n \geq 32$,

$$10 + 2\,\mathrm{bits}(n) \leq n.$$

Indeed, on the block $2^r \leq n < 2^{r+1}$ with $r \geq 5$, its left side is $12 + 2r \leq 2^r \leq n$. The inequality at $r = 5$ holds, and its preservation when r increases is immediate. The assumption $n \geq c + 2$ handles the other term in the maximum. The witness therefore belongs to $\mathcal{T}_n$, proving (13). If a fixed p denoted M_q, (13) would assert $M_q(n) < M_q(n)$ for sufficiently large n. □

The conclusion concerns the specified finite object language, not all Lean programs. The host can define M_q because it can interpret the whole type code family and recurse on syntax. An object-language term has one fixed finite type. No internal global reflection principle is inferred.

4.3 Binary visibility at a fixed budget

Replacing both unary numerals in (8) by binary numerals gives depth at most

$$\max\big(11 + 2\,\mathrm{bits}(i),\ 10 + 2\,\mathrm{bits}(n)\big). \tag{14}$$

This follows by counting the two curried applications and the outer successor: the exact cost is $\max(\mathrm{depth}(b(i)) + 3, \mathrm{depth}(b(n)) + 2)$. At $n = 64$, every $i < 2^{28}$ fits within depth 68, since the two upper bounds are at most 67 and 24. Thus

$$q(i, 64) < M_q(68) \qquad (0 \leq i < 2^{28}). \tag{15}$$

This is a sufficient bound, not an optimal coding claim. Some larger labels also fit because their binary expansions contain fewer 1s.

5 Ramified reflection on trees

Definition 5.1 (Stage functions)**.** Define $R : \mathcal{W} \to \mathbb{N} \to \mathbb{N}$ by the tree recursor:

$$R_{\mathbf{0}}(n) = M_{q_0}(n+4), \qquad q_0(i, x) = 0, \tag{16}$$

$$R_{\mathsf{sup}f}(n) = M_{q_f}(n+4), \qquad q_f(i, x) = R_{f(i)}(x). \tag{17}$$

The recursive clause is exactly the step algebra for (4) with result type $\mathbb{N} \to \mathbb{N}$. The tree children need not have a uniform numerical height bound. Their structural accessibility is supplied by the inductive type itself.

Theorem 5.2 (Constructive totality)**.** *Every R_a is total. For any closed computably constructed $a : \mathcal{W}$, the function $R_a : \mathbb{N} \to \mathbb{N}$ is computable in principle.*

Proof. The leaf algebra in (16) is total. Given the recursive functions $i \mapsto R_{f(i)}$, the step algebra merely supplies them as the binary oracle to a total finite envelope. Hence it is a total function $\mathbb{N} \to \mathbb{N}$. Applying the tree recursor gives the result. All operations involved are computational recursors or structural definitions, so a closed constructive input yields a closed computational function. □

The phrase "computably constructed" matters when interpreting the theorem in ordinary set-theoretic semantics: arbitrary mathematical functions $\mathbb{N} \to \mathcal{W}$ can carry noncomputable information. Our entry uses only the explicit closed constructions below; it does not select arbitrary semantic trees from a full set-theoretic function space.

Theorem 5.3 (Visible-child separation)**.** *For every $f : \mathbb{N} \to \mathcal{W}$ and every $i \leq n$,*

$$R_{f(i)}(n) < R_{\mathsf{sup}f}(n).$$

Also $n < R_a(n)$ for every tree a and natural n.

Proof. Apply (9) to q_f and use (17). For the second statement, apply (10) to the oracle at a, treating the leaf and supremum constructors separately. □

Corollary 5.4 (Many visible branches at input 64)**.** *For every $i < 2^{28}$, $R_{f(i)}(64) < R_{\mathsf{sup}f}(64)$.*

Proof. Use (15) with $q = q_f$. □

At each fixed tree a, Theorem 4.1 shows that R_a eventually outgrows every fixed unary program in the language using the local oracle assigned to a. This applies at the leaf as well as at nonleaf stages. It is a uniform syntactic phenomenon, not an assertion derived only from a suggested ordinal rank.

6 A quantitative amplification: oracle-fed iteration

A single reflection step can do substantially more than add one to a child value. We now prove the numerical amplification that yields (2).

6.1 A depth-14 power-of-two program

Let $D^{[l]}$ be the doubling program (11) with its accumulator variable changed from level 1 to level $l+1$, so it denotes doubling when evaluated in an environment of length l. Its depth is $l+8$. Define the closed program

$$P = \mathsf{rec}_{\mathsf{N}}\,(\mathsf{S0})\,(\lambda_{\mathsf{N},\mathsf{N}\to\mathsf{N}}\lambda_{\mathsf{N},\mathsf{N}}(D^{[2]}\,\mathsf{var}(1))). \tag{18}$$

It has depth 14 and denotes $x \mapsto 2^x$. The change to level 3 inside $D^{[2]}$ is essential: simply inserting the original D below two new binders would capture the wrong accumulator under the upstream convention.

To verify the depth, $\mathrm{depth}(D^{[2]}) = 10$, application to $\mathsf{var}(1)$ gives 11, the two outer lambdas give 13, and the final application of the partially applied recursor gives 14. Its value follows from the natural-recursion equations: start at one and double the accumulator at each step.

Lemma 6.1 (Uniform exponential lower bound)**.** *For every tree a and every $x \geq 32$,*

$$2^x \leq R_a(x).$$

Proof. The program $P\,b(x)$ is independent of the local oracle and has value 2^x. Its depth is at most $1 + \max(14, 8 + 2\,\mathrm{bits}(x)) \le x + 4$ for $x \ge 32$, by the binary inequality proved in Section 4. It therefore occurs in the envelope defining $R_a(x)$. □

6.2 The feedback witness

For a fixed binary q, write $g(x) = q(0, x)$. In the empty environment, let

$$A_q = \lambda_{\mathsf{N},\mathsf{N}\to\mathsf{N}}\lambda_{\mathsf{N},\mathsf{N}}(\mathsf{oracle}\,\mathsf{0}\,\mathsf{var}(1)).$$

The syntax A_q uses the oracle constant, not the semantic function q as an extra constructor. Its depth is 6, and it ignores its first argument while applying g to its accumulator. Define

$$F_n = \mathsf{S}\Big(\mathsf{rec}_{\mathsf{N}}\,b(n)\,A_q\,(\mathsf{oracle}\,\mathsf{0}\,b(n))\Big). \tag{19}$$

Writing $g^{[j]}$ for j-fold iteration, the recursor equations give

$$\mathrm{val}_q(F_n) = g^{[g(n)]}(n) + 1. \tag{20}$$

The two occurrences of $b(n)$ remain at top level; the only variable within the step function is the correctly scoped accumulator at level 1.

Lemma 6.2 (Feedback witness cost)**.** *The term F_n has depth at most $\max(9, 12 + 2\,\mathrm{bits}(n))$. For $n \ge 32$ this is at most $n + 4$. For the specified binary encoding, F_{64} has exact depth 20.*

Proof. Put $d = \mathrm{depth}(b(n))$. The recursor applied to its initial value has depth $1 + \max(2, d)$. Applying it to the depth-6 step gives depth $\max(7, d + 2)$. The iteration-count term $\mathsf{oracle}\,\mathsf{0}\,b(n)$ has depth $\max(3, d + 1)$. The next application and outer successor give total depth $\max(9, d + 4)$. Insert (12). For $n \ge 32$, the blockwise estimate used earlier gives the required bound. The binary representation of 64 is 1000000: its numeral has depth 16, so $\max(9, 16 + 4) = 20$. □

Theorem 6.3 (Oracle-fed iteration separation)**.** *For every $f : \mathbb{N} \to \mathcal{W}$, set $g = R_{f(0)}$. Then for every $n \ge 32$,*

$$g^{[g(n)]}(n) < R_{\mathsf{sup}f}(n). \tag{21}$$

Proof. Use $q(i, x) = R_{f(i)}(x)$ in (19). The witness has value one more than the left side by (20), and fits the stage's $n + 4$ budget by Lemma 6.2. The envelope bounds its value. □

Corollary 6.4 (Tower separation from the zeroth child)**.** *For $n \ge 32$ and $m = R_{f(0)}(n)$,*

$$E_m(n) < R_{\mathsf{sup}f}(n).$$

Proof. Lemma 6.1 shows $g(x) \ge 2^x$ on $x \ge 32$. Induction on j gives $g^{[j]}(n) \ge E_j(n)$: the base is equality, and at a successor the induction hypothesis ensures the input remains at least 32, after which $g(g^{[j]}(n)) \ge 2^{g^{[j]}(n)} \ge 2^{E_j(n)}$. Set $j = m$ and apply Theorem 6.3. □

These proofs use explicit finite terms rather than guesses about growth-rate classifications. The critical cost is the small *description* of the iteration count $g(n)$, not the size of that count.

7 Higher-type countable iteration

We next construct the tree index Θ. This adds substantial structure beyond what is needed for the basic comparison theorem.

7.1 Finite levels and pointwise suprema

Define a family of types

$$L_0 = \mathcal{W}, \qquad L_{k+1} = L_k \to L_k. \tag{22}$$

Define a countable-supremum operation at each level by

$$S_0(f) = \mathsf{sup}(f), \tag{23}$$
$$S_{k+1}(f)(x) = S_k(j \mapsto f(j)(x)). \tag{24}$$

For $u : L_k \to L_k$, let $u^{[0]}(x) = x$ and $u^{[j+1]}(x) = u(u^{[j]}(x))$. The countable-iteration operator is

$$\Omega_k(u)(x) = S_k(j \mapsto u^{[j]}(x)), \qquad \Omega_k : L_{k+2}. \tag{25}$$

The family L_k is a natural-number-indexed family of types. It is not a sequence of stronger logical axioms. Every member lies in the same small universe, and Lean can express the family by natural recursion.

For a starting tree a and an operation $s : \mathcal{W} \to \mathcal{W}$, set

$$\sigma_0 = a, \qquad \sigma_1 = s, \qquad \sigma_{k+2} = \Omega_k. \tag{26}$$

These seeds have types $\sigma_k : L_k$. Define evaluation down the levels by

$$C_0(x) = x, \qquad C_{k+1}(u) = C_k(u(\sigma_k)), \tag{27}$$

with dependence on a, s understood. Finally let

$$H_k(a; s) = C_{k+2}(\Omega_k). \tag{28}$$

Every operation here is a structural recursion on a natural number, a finite function iteration, or a tree constructor.

7.2 Structural identities and their meaning

Lemma 7.1 (Commutation with countable suprema)**.** *In extensional mathematical notation, for every $f : \mathbb{N} \to L_k$,*

$$C_k(S_k(f)) = \mathsf{sup}(j \mapsto C_k(f(j))).$$

Proof. For $k = 0$ the two sides are identical. At level $k + 1$, apply (27) and then (23) to obtain $C_k(S_k(j \mapsto f(j)(\sigma_k)))$. The induction hypothesis identifies this with $\mathsf{sup}(j \mapsto C_k(f(j)(\sigma_k)))$, which is the claimed expression by the definition of C_{k+1}. □

Proposition 7.2 (Visible predecessor in adjacent levels)**.** *One has*

$$H_0(a;s) = \mathsf{sup}(j \mapsto s^{[j]}(a)), \tag{29}$$

$$H_k(a;s) = \mathsf{sup}(j \mapsto C_k(\sigma_{k+1}^{[j]}(\sigma_k))). \tag{30}$$

In the root presentation of $H_{k+1}(a;s)$ in (30), child 1 is $H_k(a;s)$. Consequently, for $n \geq 1$,

$$R_{H_k(a;s)}(n) < R_{H_{k+1}(a;s)}(n).$$

Proof. Expanding two steps of collapse in (28) gives $C_k(\Omega_k(\sigma_{k+1})(\sigma_k))$. Expand (25) and apply Lemma 7.1 to obtain (30). Its $k = 0$ instance is (29). At level $k+1$, the child at index 1 is

$$C_{k+1}(\sigma_{k+2}(\sigma_{k+1})) = C_{k+1}(\Omega_k(\sigma_{k+1})) = H_k(a;s).$$

Theorem 5.3 now applies with child index 1. □

The general equality statements in this subsection are analysis-only extensional identities. The Lean proof source includes (29), which is definitional, but does not claim a completed empty-axiom formalization of Lemma 7.1 or the general adjacent-level identity. A pointwise logical relation is one possible way to formalize these statements without adding functional extensionality as a dependency. None of these analysis-only equalities is needed to prove the entry's required strict comparison.

8 Numerical feedback into type-level indices

Define a simple tree successor operation

$$\mathsf{Step}(a) = \mathsf{sup}(i \mapsto a).$$

Unlike a quotient ordinal successor, this is merely an explicit tree with the same child at every index. The name serves as a structural mnemonic.

Definition 8.1 (Feedback boost)**.** Define

$$\mathsf{Boost}(a) = \mathsf{sup}(b_a), \qquad b_a(0) = a, \quad b_a(k+1) = H_{R_a(k+64)}(a;\mathsf{Step}). \tag{31}$$

The construction feeds a reflected *number* into the height of the finite-type family (22). This is stronger structurally than choosing a larger fixed literal in a single envelope. It does not rely on an assumption that this type height has a particular ordinal name.

There is no circular definition. The operation R was completed in Section 5 before Boost was introduced. The body of Boost calls that already defined operation on its input a. The functions H_k were likewise defined for all natural k before any $R_a(k+64)$ was used as their index.

Theorem 8.2 (Feedback comparisons)**.** *For every a and every n,*

$$R_a(n) < R_{\mathsf{Boost}(a)}(n).$$

For $n \geq 1$ one also has

$$R_{H_{R_a(64)}(a;\mathsf{Step})}(n) < R_{\mathsf{Boost}(a)}(n).$$

Proof. The retained branch at index 0 gives the first inequality via Theorem 5.3. Branch 1 is exactly $H_{R_a(64)}(a;\mathsf{Step})$, giving the second. No comparison between the numerical oracles as pointwise functions is required. □

Definition 8.3 (The grand index and proposed entry)**.** Set

$$\Theta = \mathsf{sup}(k \mapsto H_k(\mathbf{0};\mathsf{Boost})), \qquad \mathfrak{N} = R_\Theta(64). \tag{32}$$

Theorem 8.4 (Some explicit stages exceeded by the entry)**.** *For $k \le n$,*

$$R_{H_k(\mathbf{0};\mathsf{Boost})}(n) < R_\Theta(n).$$

For $j \le n$,

$$R_{\mathsf{Boost}^{[j]}(\mathbf{0})}(n) < R_\Theta(n).$$

At $n = 64$, both conclusions hold for every $k < 2^{28}$ and every $j < 2^{28}$, respectively.

Proof. The first statement uses child k of (32). The second uses child j of $H_0(\mathbf{0};\mathsf{Boost})$ in (29), followed by child 0 of Θ, and composes the two strict inequalities. For input 64, use Corollary 5.4 instead of the unary-index version at the indicated child selections. □

The word "all" in "all finite levels" describes the index construction: Θ contains a child for each natural level. A fixed numerical input has a fixed syntax budget. The theorem carefully states which branch comparisons that budget proves, rather than claiming that one fixed output exceeds an unbounded sequence of natural outputs at that input.

9 The exact comparison with Contender 5

Theorem 9.1 (Uniform comparison with the upstream baseline)**.** *For every total binary oracle q,*

$$C_5 < M_q(68).$$

Consequently, for every tree a,

$$C_5 < R_a(64).$$

Proof. By Lemma 2.1, $t_* \in T_{42}$. The quotation block of (6) places $\mathsf{quote}(t_*)$ in $\mathcal{T}_{43}$. Both this term and S therefore occur at depth 43, so the application

$$e_* = \mathsf{S}(\mathsf{quote}(t_*))$$

occurs in $\mathcal{T}_{44}$. Cumulativity puts it in $\mathcal{T}_{68}$. Its value is $\mathrm{val}_0(t_*) + 1 = C_5 + 1$, independently of q. Thus $C_5 + 1 \le M_q(68)$. For any tree a, its stage at 64 is exactly such an envelope, since $64 + 4 = 68$; distinguish the two constructors in (16)–(17). □

Corollary 9.2 (Required strict inequality)**.** *The proposed number is strictly greater than the pinned upstream Contender 5: $C_5 < \mathfrak{N}$.*

Proof. Take $a = \Theta$ in Theorem 9.1. □

This proof does not evaluate the old maximum or the new one. It proves that a particular *description* occurs in the relevant finite list. The successor operation appears in that proof witness, not as the definition of the new entry. Neither the literal 42, the old maximum function, nor the constant `contender_5` occurs in `Hierarchy.lean` or `Contender.lean`.

Theorem 9.3 (Tower-height separation)**.** *Let* $E_0(x) = x$ *and* $E_{j+1}(x) = 2^{E_j(x)}$*. Then*

$$E_{C_5+1}(64) < \mathfrak{N}. \tag{33}$$

In particular, the separation is far stronger than a fixed number of successor steps or a single exponential in C_5*.*

Proof. Write $a_0 = H_0(\mathbf{0}; \mathsf{Boost})$ for the zeroth child of Θ, and set $m = R_{a_0}(64)$. By Theorem 9.1, $m > C_5$, hence $m \geq C_5 + 1$. Corollary 6.4 applied to the root of Θ gives

$$E_m(64) < R_\Theta(64) = \mathfrak{N}.$$

For $x \geq 1$, the sequence $E_j(x)$ is increasing: $E_1(x) = 2^x > x$, and successive exponentiations preserve the strict inequality. Therefore $E_{C_5+1}(64) \leq E_m(64) < \mathfrak{N}$. □

The tower theorem uses only a small part of the grand index; it is a robust lower bound, not an estimate of the entry's actual size. Most of the higher-type and feedback construction is deliberately not unfolded by the comparison proof.

10 Why the construction is constructive rather than a loophole

10.1 No Busy Beaver and no unrestricted Lean evaluator

Busy-Beaver-style maximization selects outputs of programs whose termination cannot in general be determined by a total evaluator. Here every raw term in each finite list already has a total denotation, established before the maximum is defined. The evaluator does not run arbitrary Lean source, search for consistency proofs, assume reflection for Lean itself, or recognize a set of all total computable functions. The fixed-language escape result is exactly why a universal self-maximizing interpretation would be inappropriate.

10.2 Infinite branching does not mean nontermination

The only infinite object is a function giving children on demand. The tree fold receives a function of recursive results and may inspect finitely many children along a particular computation. A program supplied to the fold can make the number of such inspections extraordinarily large, but its type and recursor-based construction are what justify totality. There is no infinite while-loop hidden behind a supposedly finite search bound.

10.3 Avoiding an invalid oracle-monotonicity argument

An object program can react nonmonotonically to its oracle inputs. Therefore one should not infer, merely from $q \leq q'$ pointwise, that every program's output under q is at most its output under q'. The proof strategy here never needs such a statement. Envelope monotonicity is in the *syntax cutoff*, and stage comparisons use a specific term that calls the specified child oracle. The tower theorem likewise uses a known program, not a global monotonicity claim about interpreting every raw term.

10.4 No unwarranted ordinal comparison

The constructor `sup` retains its branch order and syntactic presentation. Two trees with equal externally assigned ordinal ranks may expose different children at low numerical costs. Pointwise stage inequalities are proved from those accessible children. An ordinal-ranking theorem, if developed, would be additional work and would not replace the explicit coding estimates needed for this contest.

10.5 No claim of absolute optimality

One can always define a successor or another reflection layer after a fixed entry. The contest's computational time bounds and its prohibition of lazy references shape a practical research problem, not a canonical greatest natural number. Our proposal fixes a substantial language extension and repeated higher-type amplification, and supplies a proof of a large margin. It does not pretend that no more effective idea could be found with further research.

11 Lean artifact architecture

The definition path is deliberately separate from the baseline used in the proof:

$$\texttt{ListKit} \longrightarrow \texttt{Legacy} \longrightarrow \texttt{Language} \longrightarrow \texttt{Hierarchy} \longrightarrow \texttt{Contender},$$
$$\texttt{Legacy} \longrightarrow \texttt{Baseline}, \qquad (\texttt{Contender}, \texttt{Baseline}) \longrightarrow \texttt{Proofs}.$$

`Audit` and `Smoke` are downstream validation modules. `Legacy` supplies the old raw evaluator, not its winning constant; `Baseline` alone supplies that constant.

Module	Responsibility
`ListKit.lean`	Elementary list operations, lookup, maximum, argmax.
`Legacy.lean`	Computational Coq-semantics port, weighted depth, raw enumerators.
`Language.lean`	Tree type, total extended interpreter, quotation, cumulative enumeration.
`Hierarchy.lean`	R, finite type levels, pointwise suprema, collapse, feedback, Θ.
`Contender.lean`	The single proposed entry $R_\Theta(64)$.
`Baseline.lean`	The exact selected-term definition of C_5.
`Proofs.lean`	Occurrence lemmas, basic child and feedback inequalities, strict baseline proof.
`Audit.lean`	Transitive axiom queries for the entry and central theorems.
`Smoke.lean`	Tiny definitional examples, not giant-number evaluation.

The maximum proof uses an explicit inductive list-occurrence predicate. This avoids dependence on extensional finite-set APIs and makes the needed membership derivations short. The final proof is intended to elaborate to an application of the uniform baseline theorem, rather than normalize Θ or run a maximum. The definition itself uses structural recursors and does not employ a noncomputable semantic supremum.

The code's size is not evidence that the time limits are met. In particular, Lean elaboration, generated code for recursors, imported theorem dependencies, and conversion checking all require an actual toolchain run. These are precisely what the acceptance script is designed to test.

12 Validation performed and validation still required

12.1 Executed small-instance checks

The Python model is an independent executable rendition of the raw semantics, with explicit resource guards. It refuses large enumerations rather than substituting bounded or approximate values into the mathematical definition. It is not a Lean checker and does not certify totality.

The recorded run passed **62,065 assertions**. The old-language enumeration results were:

Depth	Raw terms	Types	Maximum numerical output
1	2	1	0
2	10	2	1
3	146	5	2
4	24,976	26	3

For the new language, take $q_0(i,n) = 0$, $q_1(i,n) = 3i + n + 7$, and $q_2(i,n) = \max(17 - i - n, 0)$:

Depth	Raw terms	Types	M_{q_0}	M_{q_1}	M_{q_2}
1	5	2	0	0	0
2	62	6	1	1	1
3	6,167	38	2	8	17

The tiny results are not approximations to C_5 or $\mathfrak{N}$.

The tests additionally cover argmax/maximum agreement, selection membership, quotation and successor semantics, all enumerated depth bounds, cumulative inclusion, absolute binder levels, malformed applications and casts, binary numerals, the explicit depth-8 doubling and depth-14 exponentiation programs, the depth-20 feedback witness at input 64, oracle-witness semantics, and finite observations of higher-type branch identities. Binary inequalities are checked through 9,999 in addition to their general mathematical proof. The model tests power-of-two semantics only at small inputs and feedback semantics only for small simple oracles; it never evaluates a giant stage.

12.2 Unexecuted Lean validation

There was no `lean`, `lake`, or `elan` executable in the preparation container. Attempts to obtain a toolchain did not succeed. The validation report therefore records

```
"kernel_checked": false,
"timing_qualified": false,
"status": "not_run_compiler_unavailable"
```

A preliminary lexical source screen found no prohibited tokens in code after stripping comments and strings. This is only a source screen. It cannot establish elaboration, kernel acceptance, or the transitive axiom set of a compiled theorem. The intended strict comparison proof contains no placeholder, but unobserved syntax, API, termination-elaboration, simplifier, or code-generation issues may remain.

Claim or check	Delivered status
Constructive mathematical definition	Fully specified
Mathematical proof $C_5 < \mathfrak{N}$	Provided
Mathematical tower-height separation	Provided
Exact computational port argument	Provided; no cross-kernel certificate
Lean source for basic comparison	Provided; uncompiled
Lean source for tower/escape theorems	Not formalized in this package
Finite Python model	62,065 assertions passed
Source placeholder screen	Passed, limited scope
Lean kernel and empty-axiom audit	Not run
15-second definition / 60-second proof	Unverified
Accepted contest submission	Not claimed

12.3 Reproduction and acceptance gates

From the extracted archive root, the model and document are reproducible by

```
python python/test_model.py
bash scripts/build_article.sh
```

On a machine with the pinned Lean toolchain installed, run

```
python scripts/verify_lean.py
```

The script removes this package's generated Lean module artifacts, checks its source screen, and compiles the five definition modules against the prebuilt standard distribution under an aggregate 15-second budget. It then compiles the baseline, proof, audit, and smoke modules under an aggregate 60-second budget. It records compiler output, elapsed times, and axiom reports. Any compiler error, missing empty-axiom result, or time-budget failure prevents an accepted status.

The required audit includes both the entry and its strict comparison:

```
BigNumber.contender_6
BigNumber.contender_5_lt_contender_6
```

An empty dependency list is required, not merely absence of a user-written `axiom` declaration. The script's parser and timing policy are themselves ordinary validation utilities, not a security-certified sandbox. A contest judge should also review the source, compiler provenance, and machine description.

Do not run `#eval contender_6`, and do not attempt to print its decimal expansion. The problem asks for a checked total definition and comparison, not feasible normalization of the output.

13 Further research questions

Complete and independently replay the strict formalization. The immediate concrete task is to run the pinned compiler, repair any elaboration issues without weakening the mathematical

specification, and obtain empty axiom reports and disclosed timings. Rechecking with an independent kernel implementation would strengthen the trust story. The cross-language port should ultimately receive a machine-checked semantic simulation rather than remain a paper argument plus finite tests.

Formalize the tower engine and the binary coding theorem. The strongest quantitative result is presently proved on paper. A systematic library of context-aware quotation and depth certificates would make the power-of-two and feedback witnesses straightforward to verify without normalizing their outputs. Absolute binder levels make capture-avoiding lifting an especially important component.

Optimize visible-child coding. The bound 2^{28} is deliberately conservative. Exact depth depends on both bit length and population count in our binary encoding. Better addition chains, shared descriptions, or a different explicitly charged numeral language could improve it. The relevant optimization target is guaranteed witness depth, not an informal textual abbreviation.

Replace extensional tree equalities by logical relations. Formalize the higher-type supremum/collapse identities in a way compatible with the strict empty-axiom rule. A family of pointwise relations indexed by finite type codes may prove observational equality without appealing to functional extensionality as an axiom. One must also prove the evaluator preserves that relation.

Determine genuine growth classifications. How does the finite-language envelope with W-recursion compare with named fast-growing hierarchies under a specified coding? What are sharp lower and upper bounds for the finite-type diagonal and feedback operation? An answer requires precise ordinal notation or another semantic ranking together with coding estimates. No identification with a famous ordinal is assumed here.

Investigate stronger constructive object languages. The upstream repository also includes work toward a System F evaluator [4]. Possible directions include a carefully stratified polymorphic language, inductive families, or richer tree constructors, but each extension must retain a total executable interpreter and satisfy the actual definition-time gate. A larger semantic language is useful only when its interpreter is genuinely specified and justified.

Separate semantic improvement from packaging tricks. A useful research standard would compare constructors, available recursion principles, and budgeted embedding theorems, rather than reward hidden precompiled definitions or unconstrained source abbreviation. The present entry suggests measuring language inclusion and explicit stage simulation alongside the traditional numerical comparison.

14 Conclusion

The proposed number is a closed constructive expression with a substantial structural design: finite-language maximization, tree-indexed reflection, arbitrary finite higher-type iteration, and feedback of semantic values into type-level indices. Its relation to the actual upstream entry is

not left to growth-rate intuition. The successor-quotation witness proves $C_5 < \mathfrak{N}$, and a separate depth-20 iteration witness proves the much stronger inequality $E_{C_5+1}(64) < \mathfrak{N}$.

The package makes the remaining boundary explicit. It provides mathematical proofs, proposed Lean source, tests, and acceptance tooling. It does not provide a successful Lean kernel run or a timing certificate. Thus it is a serious research contender ready for the stated validation work, rather than an already certified winner.

A The central Lean definition, in ASCII notation

The following is a readable ASCII rendering of the central definitions; the actual source files are authoritative. It uses the interpreter and finite envelope from `Language.lean`.

```
def reflect : Tree -> Nat -> Nat :=
  Tree.rec
    (fun n => Language.envelope (fun _ _ => 0) (n + 4))
    (fun _ ih n =>
      Language.envelope (fun i x => ih i x) (n + 4))

def Level : Nat -> Type
  | 0 => Tree
  | n + 1 => Level n -> Level n

def supLevel : (k : Nat) -> (Nat -> Level k) -> Level k
  | 0, f => Tree.sup f
  | k + 1, f => fun x => supLevel k (fun n => f n x)

def omegaLevel (k : Nat) : Level (k + 2) :=
  fun f x => supLevel k (fun n => iterate f n x)

def seed (a : Tree) (s : Tree -> Tree) : (k : Nat) -> Level k
  | 0 => a
  | 1 => s
  | k + 2 => omegaLevel k

def collapse (a : Tree) (s : Tree -> Tree) :
    (k : Nat) -> Level k -> Tree
  | 0, x => x
  | k + 1, f => collapse a s k (f (seed a s k))

def high (a : Tree) (s : Tree -> Tree) (k : Nat) : Tree :=
  collapse a s (k + 2) (omegaLevel k)

def step (a : Tree) : Tree := Tree.sup (fun _ => a)

def boost (a : Tree) : Tree :=
  Tree.sup (fun
    | 0 => a
    | k + 1 => high a step (reflect a (k + 64)))

def grandIndex : Tree :=
  Tree.sup (fun k => high Tree.zero boost k)

def contender_6 : Nat := reflect grandIndex 64
```

Here `iterate` is ordinary natural-number recursion on the iteration count. There is no call to the previous winning number in this definition.

B Provenance and reproducibility notes

The upstream snapshot is pinned by:

Repository commit `d56dce013e6264292533efdc5e1de71863c6fb86`.

Contender.v blob `f92db72deb0f3a4d4e3b575b4949eda73b1417a0`.

README.md blob `b2a2642114e50e86f4432f59aebf439617f78d18`.

The ported computational material retains attribution under the upstream MIT license; its complete notice is in `provenance/UPSTREAM_LICENSE.txt`. The package includes source metadata, not a complete upstream repository mirror. No toolchain executable, font file, or giant decimal numeral is included.

The article's test counts correspond to `validation/python_tests.json`. The unverified Lean status is recorded in `validation/lean_validation.json`. Re-running the scripts overwrites these reports with the new observed results; such a run should be archived with its machine details if used for contest adjudication.

References

[1] Cody Roux and contributors, *name-the-biggest-number*, source repository, accessed 12 October 2026. https://github.com/codyroux/name-the-biggest-number.

[2] Cody Roux, *name-the-biggest-number: README and contest rules*, pinned snapshot described in Appendix B. https://github.com/codyroux/name-the-biggest-number/blob/d56dce013e6264292533efdc5e1de71863c6fb86/README.md.

[3] Cody Roux, Samuel Grütter, and contributors, *Contender.v*, especially `cast_impl`, `interp_term`, `termsUpTo`, `maxBy`, and `contender_5`, pinned snapshot. https://github.com/codyroux/name-the-biggest-number/blob/d56dce013e6264292533efdc5e1de71863c6fb86/Contender.v.

[4] Cody Roux, *System_F.v*, source in the same repository. Only the opening normalization/evaluator development was inspected for this report. https://github.com/codyroux/name-the-biggest-number/blob/d56dce013e6264292533efdc5e1de71863c6fb86/System_F.v.

[5] Lean contributors, *The Lean Language Reference: Inductive Types*, accessed 12 October 2026. https://lean-lang.org/doc/reference/latest/The-Type-System/Inductive-Types/.

[6] Lean contributors, *The Lean Language Reference: Recursive Definitions*, accessed 12 October 2026. https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/.

[7] Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich, *Theorem Proving in Lean 4: Axioms and Computation*, accessed 12 October 2026. https://lean-lang.org/theorem_proving_in_lean4/Axioms-and-Computation/.

[8] Lean contributors, *Lean v4.35.0 release*, selected toolchain target; not executed in the preparation environment. https://github.com/leanprover/lean4/releases/tag/v4.35.0.

[9] Scott Aaronson, *Who Can Name the Bigger Number?*, background essay linked by the contest repository. https://www.scottaaronson.com/writings/bignumbers.html.